Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №4

з дисципліни

«ОБ'ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ»

на тему:

«Вдосконалення структури коду графічного редактора об'єктів»

Виконала:
Студентка групи ІМ-54
Дрига О.О
Номер в списку: 11

Перевірив:
Рекечинський Д.О.

Київ 2026

Завдання:

1. Створити у середовищі розробки проект з ім'ям Lab4.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. Налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.

Завдання згідно з варіантом (номер у списку 11):

- **Об'єкт класу MyEditor (11 - непарний номер):** глобальний статичний об'єкт класу MyEditor (static MyEditor editor;).
- **Кольори та стилі фігур:** як у лабораторній роботі №3 (Ж = 12):
  - масив вказівників - динамічний Shape **pcshape, N = 112;
  - прямокутник вводиться по двох протилежних кутах, чорний контур із жовтим заповненням;
  - еліпс вводиться від центру до кута охоплюючого прямокутника, чорний контур без заповнення;
  - позначка поточного об'єкта відображається в меню (метод OnInitMenuPopup).
- **«Гумовий» слід:** пунктирна лінія для всіх фігур.
- **Дві нові фігури:** лінія з кружечками та каркас куба.
  - Лінія з кружечками - клас LineWithCircles, множинне успадкування від LineShape та EllipseShape. Червона лінія, кружечки зі світло-рожевим заповненням.
  - Каркас куба - клас CubeShape, множинне успадкування від LineShape та RectangleShape. Синій каркас без заповнення.
  - Метод Show нових фігур викликає Show базових класів: LineShape::Show для ліній, EllipseShape::Show для кружечків, RectangleShape::Show для прямокутників.
- **Інтерфейс:** панель Toolbar із шістьма кнопками-іконками та підказками (tooltips), які дублюють підпункти меню «Об'єкти».
- У звіті обов'язково має бути представлена схема успадкування класів (діаграма класів).

Вихідний текст головного файлу:

main.cpp

```
#define NOMINMAX
#include <windows.h>
#include <windowsx.h>
#include "MyEditor.h"
#include "Toolbar.h"


static MyEditor editor;


class MainWindow {
    HWND      hWnd;
    HINSTANCE hInst;
    HMENU     hObjectsMenu;
    Toolbar   toolbar;


    HMENU CreateAppMenu() {
        HMENU hMenu = CreateMenu();
        HMENU hFileMenu = CreatePopupMenu();
        hObjectsMenu = CreatePopupMenu();
        HMENU hHelpMenu = CreatePopupMenu();


        AppendMenuW(hFileMenu, MF_STRING, IDM_EXIT, L"Вихід");


        AppendMenuW(hObjectsMenu, MF_STRING, IDM_POINT, L"Крапка");
        AppendMenuW(hObjectsMenu, MF_STRING, IDM_LINE, L"Лінія");
        AppendMenuW(hObjectsMenu, MF_STRING, IDM_RECT, L"Прямокутник");
        AppendMenuW(hObjectsMenu, MF_STRING, IDM_ELLIPSE, L"Еліпс");
        AppendMenuW(hObjectsMenu, MF_STRING, IDM_LINECIRCLE, L"Лінія з кружечками");
        AppendMenuW(hObjectsMenu, MF_STRING, IDM_CUBE, L"Каркас куба");


        AppendMenuW(hHelpMenu, MF_STRING, IDM_ABOUT, L"Про програму");


        AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hFileMenu, L"Файл");
        AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hObjectsMenu, L"Об'єкти");
        AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hHelpMenu, L"Довідка");
        return hMenu;
    }


    LRESULT OnCreate() {
        SetMenu(hWnd, CreateAppMenu());
        toolbar.Create(hWnd, hInst);
        return 0;
    }


    LRESULT OnSize() {
        toolbar.AutoSize();
        return 0;
    }


    LRESULT OnCommand(WPARAM wParam) {
        int id = LOWORD(wParam);
        switch (id) {
        case IDM_POINT:
        case IDM_LINE:
        case IDM_RECT:
        case IDM_ELLIPSE:
        case IDM_LINECIRCLE:
        case IDM_CUBE:
            editor.SetType(id);
            toolbar.Check(id);
            break;
        case IDM_ABOUT:
            MessageBoxW(hWnd,
                L"Лабораторна робота №4\nВаріант №11\nВиконала: студентка Дрига Олександра ІМ-
54",
                L"Про програму", MB_OK | MB_ICONINFORMATION);
            break;
        case IDM_EXIT:
            DestroyWindow(hWnd);
```

```
        break;
    }
    return 0;
}

LRESULT OnNotify(LPARAM lParam) {
    LPNMHDR hdr = (LPNMHDR)lParam;
    if (hdr->code == TTN_GETDISPINFOW) {
        LPNMTTDISPINFOW tip = (LPNMTTDISPINFOW)lParam;
        tip->lpszText = (LPWSTR)Toolbar::TipText((int)hdr->idFrom);
    }
    return 0;
}

LRESULT OnInitMenuPopup(WPARAM wParam, LPARAM lParam) {
    HMENU hMenu = (HMENU)wParam;
    if (HIWORD(lParam) == 0 && hMenu == hObjectsMenu) {
        CheckMenuRadioItem(hMenu, IDM_POINT, IDM_CUBE,
                           editor.GetType(), MF_BYCOMMAND);
    }
    return 0;
}

LRESULT OnLButtonDown(LPARAM lParam) {
    POINT pt = { GET_X_LPARAM(lParam), GET_Y_LPARAM(lParam) };
    editor.OnLButtonDown(hWnd, pt);
    return 0;
}

LRESULT OnMouseMove(LPARAM lParam) {
    POINT pt = { GET_X_LPARAM(lParam), GET_Y_LPARAM(lParam) };
    editor.OnMouseMove(hWnd, pt);
    return 0;
}

LRESULT OnLButtonUp(LPARAM lParam) {
    POINT pt = { GET_X_LPARAM(lParam), GET_Y_LPARAM(lParam) };
    editor.OnLButtonUp(hWnd, pt);
    return 0;
}

LRESULT OnPaint() {
    PAINTSTRUCT ps;
    HDC hdc = BeginPaint(hWnd, &ps);
    editor.OnPaint(hdc);
    EndPaint(hWnd, &ps);
    return 0;
}

LRESULT OnDestroy() {
    PostQuitMessage(0);
    return 0;
}

LRESULT HandleMessage(UINT msg, WPARAM wParam, LPARAM lParam) {
    switch (msg) {
    case WM_CREATE:         return OnCreate();
    case WM_SIZE:           return OnSize();
    case WM_COMMAND:        return OnCommand(wParam);
    case WM_NOTIFY:         return OnNotify(lParam);
    case WM_INITMENUPOPUP:  return OnInitMenuPopup(wParam, lParam);
    case WM_LBUTTONDOWN:    return OnLButtonDown(lParam);
    case WM_MOUSEMOVE:      return OnMouseMove(lParam);
    case WM_LBUTTONUP:      return OnLButtonUp(lParam);
    case WM_PAINT:          return OnPaint();
    case WM_DESTROY:        return OnDestroy();
    }
    return DefWindowProcW(hWnd, msg, wParam, lParam);
}
```

```
    static LRESULT CALLBACK WndProc(HWND hWnd, UINT msg, WPARAM wParam, LPARAM lParam) {
        MainWindow* self = (MainWindow*)GetWindowLongPtrW(hWnd, GWLP_USERDATA);
        if (msg == WM_NCCREATE) {
            self = (MainWindow*)((LPCREATESTRUCTW)lParam)->lpCreateParams;
            self->hWnd = hWnd;
            SetWindowLongPtrW(hWnd, GWLP_USERDATA, (LONG_PTR)self);
        }
        if (self) return self->HandleMessage(msg, wParam, lParam);
        return DefWindowProcW(hWnd, msg, wParam, lParam);
    }

public:
    MainWindow(HINSTANCE h) : hWnd(NULL), hInst(h), hObjectsMenu(NULL) {}

    bool Create(int nCmdShow) {
        const wchar_t cls[] = L"Lab4WindowClass";

        WNDCLASSEXW wc = { 0 };
        wc.cbSize = sizeof(wc);
        wc.style = CS_HREDRAW | CS_VREDRAW;
        wc.lpfnWndProc = WndProc;
        wc.hInstance = hInst;
        wc.hCursor = LoadCursor(NULL, IDC_ARROW);
        wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
        wc.lpszClassName = cls;
        if (!RegisterClassExW(&wc)) return false;

        HWND h = CreateWindowExW(0, cls, L"Lab4 - Графічний редактор",
            WS_OVERLAPPEDWINDOW | WS_CLIPCHILDREN,
            CW_USEDEFAULT, CW_USEDEFAULT, 800, 600,
            NULL, NULL, hInst, this);
        if (!h) return false;

        ShowWindow(h, nCmdShow);
        UpdateWindow(h);
        return true;
    }

    int Run() {
        MSG msg;
        while (GetMessage(&msg, NULL, 0, 0)) {
            TranslateMessage(&msg);
            DispatchMessage(&msg);
        }
        return (int)msg.wParam;
    }
};

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE, LPSTR, int nCmdShow) {
    MainWindow app(hInstance);
    if (!app.Create(nCmdShow)) return 0;
    return app.Run();
}
```

myeditor.h

```
#ifndef MYEDITOR_H
#define MYEDITOR_H

#include "Shapes.h"
#include "Ids.h"

class MyEditor {
    static const int N = 112;

    Shape** pcshape;
```

```
    int     shapeCount;
    int     currentType;
    bool    isDrawing;
    POINT   ptStart, ptCurrent;


    Shape* CreateShape(int type) const {
        switch (type) {
        case IDM_POINT:      return new PointShape();
        case IDM_LINE:       return new LineShape();
        case IDM_RECT:       return new RectangleShape();
        case IDM_ELLIPSE:    return new EllipseShape();
        case IDM_LINECIRCLE: return new LineWithCircles();
        case IDM_CUBE:       return new CubeShape();
        }
        return nullptr;
    }


    void DrawRubber(HWND hWnd, POINT p1, POINT p2) {
        Shape* s = CreateShape(currentType);
        if (!s) return;
        s->SetRubber(true);
        s->Set(p1.x, p1.y, p2.x, p2.y);


        HDC hdc = GetDC(hWnd);
        SetROP2(hdc, R2_NOTXORPEN);
        SetBkMode(hdc, TRANSPARENT);
        s->Show(hdc);
        ReleaseDC(hWnd, hdc);
        delete s;
    }


    void AddShape(HWND hWnd, Shape* s, POINT p1, POINT p2) {
        if (!s) return;
        if (shapeCount < N) {
            s->Set(p1.x, p1.y, p2.x, p2.y);
            pcshape[shapeCount++] = s;
            InvalidateRect(hWnd, NULL, TRUE);
        }
        else {
            delete s;
        }
    }


public:
    MyEditor() : shapeCount(0), currentType(IDM_POINT), isDrawing(false) {
        pcshape = new Shape * [N];
        for (int i = 0; i < N; i++) pcshape[i] = nullptr;
        ptStart.x = ptStart.y = ptCurrent.x = ptCurrent.y = 0;
    }


    ~MyEditor() {
        for (int i = 0; i < shapeCount; i++) delete pcshape[i];
        delete[] pcshape;
    }


    MyEditor(const MyEditor&) = delete;
    MyEditor& operator=(const MyEditor&) = delete;


    void SetType(int type) { currentType = type; }
    int  GetType() const   { return currentType; }


    void OnLButtonDown(HWND hWnd, POINT pt) {
        ptStart = pt;
        ptCurrent = pt;


        if (currentType == IDM_POINT) {
            AddShape(hWnd, CreateShape(currentType), pt, pt);
            return;
```

```
        }
        isDrawing = true;
        SetCapture(hWnd);
        DrawRubber(hWnd, ptStart, ptCurrent);
    }

    void OnMouseMove(HWND hWnd, POINT pt) {
        if (!isDrawing) return;
        DrawRubber(hWnd, ptStart, ptCurrent);
        ptCurrent = pt;
        DrawRubber(hWnd, ptStart, ptCurrent);
    }

    void OnLButtonUp(HWND hWnd, POINT pt) {
        if (!isDrawing) return;
        DrawRubber(hWnd, ptStart, ptCurrent);
        isDrawing = false;
        ReleaseCapture();
        ptCurrent = pt;
        AddShape(hWnd, CreateShape(currentType), ptStart, ptCurrent);
    }

    void OnPaint(HDC hdc) const {
        for (int i = 0; i < shapeCount; i++)
            if (pcshape[i]) pcshape[i]->Show(hdc);
    }
};

#endif
```

shape.h

```
#ifndef SHAPE_H
#define SHAPE_H

#ifndef NOMINMAX
#define NOMINMAX
#endif
#include <windows.h>
#include <algorithm>
#include <cstdlib>

class Shape {
protected:
    long     xs1, ys1, xs2, ys2;
    COLORREF penColor;
    bool     rubber;

    HPEN MakePen() const {
        return rubber ? CreatePen(PS_DOT, 1, RGB(0, 0, 0))
                      : CreatePen(PS_SOLID, 1, penColor);
    }

public:
    Shape() : xs1(0), ys1(0), xs2(0), ys2(0), penColor(RGB(0, 0, 0)), rubber(false) {}
    virtual ~Shape() {}

    virtual void Set(long x1, long y1, long x2, long y2) {
        xs1 = x1; ys1 = y1;
        xs2 = x2; ys2 = y2;
    }

    void SetRubber(bool r) { rubber = r; }

    virtual void Show(HDC hdc) = 0;
};
```

```
#endif
```

shapes.h

```
#ifndef SHAPES_H
#define SHAPES_H


#include "Shape.h"


class PointShape : public Shape {
public:
    void Show(HDC hdc) override {
        SetPixel(hdc, xs1, ys1, RGB(0, 0, 0));
        MoveToEx(hdc, xs1 - 1, ys1, NULL); LineTo(hdc, xs1 + 2, ys1);
        MoveToEx(hdc, xs1, ys1 - 1, NULL); LineTo(hdc, xs1, ys1 + 2);
    }
};


class LineShape : public virtual Shape {
public:
    void Show(HDC hdc) override {
        HPEN hPen = MakePen();
        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);


        MoveToEx(hdc, xs1, ys1, NULL);
        LineTo(hdc, xs2, ys2);


        SelectObject(hdc, hOldPen);
        DeleteObject(hPen);
    }
};


class RectangleShape : public virtual Shape {
protected:
    bool     filled;
    COLORREF fillColor;
public:
    RectangleShape() : filled(true), fillColor(RGB(255, 255, 0)) {}


    void Show(HDC hdc) override {
        long left   = std::min(xs1, xs2);
        long top    = std::min(ys1, ys2);
        long right  = std::max(xs1, xs2);
        long bottom = std::max(ys1, ys2);


        bool useFill = filled && !rubber;
        HPEN   hPen   = MakePen();
        HBRUSH hBrush = useFill ? CreateSolidBrush(fillColor)
                                : (HBRUSH)GetStockObject(NULL_BRUSH);


        HPEN   hOldPen   = (HPEN)SelectObject(hdc, hPen);
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);


        Rectangle(hdc, left, top, right, bottom);


        SelectObject(hdc, hOldPen);
        SelectObject(hdc, hOldBrush);
        DeleteObject(hPen);
        if (useFill) DeleteObject(hBrush);
    }
};


class EllipseShape : public virtual Shape {
protected:
    bool     filled;
    COLORREF fillColor;
public:
```

```
    EllipseShape() : filled(false), fillColor(RGB(255, 255, 255)) {}

    void Show(HDC hdc) override {
        long dx = std::abs(xs2 - xs1);
        long dy = std::abs(ys2 - ys1);

        bool useFill = filled && !rubber;
        HPEN   hPen   = MakePen();
        HBRUSH hBrush = useFill ? CreateSolidBrush(fillColor)
                                : (HBRUSH)GetStockObject(NULL_BRUSH);

        HPEN   hOldPen   = (HPEN)SelectObject(hdc, hPen);
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

        Ellipse(hdc, xs1 - dx, ys1 - dy, xs1 + dx, ys1 + dy);

        SelectObject(hdc, hOldPen);
        SelectObject(hdc, hOldBrush);
        DeleteObject(hPen);
        if (useFill) DeleteObject(hBrush);
    }
};

class LineWithCircles : public LineShape, public EllipseShape {
public:
    LineWithCircles() {
        penColor  = RGB(200, 0, 0);
        filled    = true;
        fillColor = RGB(255, 200, 200);
    }

    void Show(HDC hdc) override {
        const long x1 = xs1, y1 = ys1, x2 = xs2, y2 = ys2;
        const long R = 6;

        LineShape::Show(hdc);

        Set(x1, y1, x1 + R, y1 + R);
        EllipseShape::Show(hdc);
        Set(x2, y2, x2 + R, y2 + R);
        EllipseShape::Show(hdc);

        Set(x1, y1, x2, y2);
    }
};

class CubeShape : public LineShape, public RectangleShape {
public:
    CubeShape() {
        penColor = RGB(0, 0, 200);
        filled   = false;
    }

    void Show(HDC hdc) override {
        const long x1 = xs1, y1 = ys1, x2 = xs2, y2 = ys2;
        long l = std::min(x1, x2), t = std::min(y1, y2);
        long r = std::max(x1, x2), b = std::max(y1, y2);
        long d = std::min(r - l, b - t) / 3;

        Set(l + d, t, r, b - d);   RectangleShape::Show(hdc);
        Set(l, t + d, r - d, b);   RectangleShape::Show(hdc);

        Set(l, t + d, l + d, t);       LineShape::Show(hdc);
        Set(r - d, t + d, r, t);       LineShape::Show(hdc);
        Set(l, b, l + d, b - d);       LineShape::Show(hdc);
        Set(r - d, b, r, b - d);       LineShape::Show(hdc);
```

```
        Set(x1, y1, x2, y2);
    }
};

#endif
```

toolbar.h

```
#ifndef TOOLBAR_H
#define TOOLBAR_H

#ifndef _WIN32_IE
#define _WIN32_IE 0x0501
#endif

#include <windows.h>
#include <commctrl.h>
#include "Ids.h"

#ifndef BTNS_CHECKGROUP
#define BTNS_CHECKGROUP 0x0006
#endif

#ifdef _MSC_VER
#pragma comment(lib, "comctl32.lib")
#endif

class Toolbar {
    HWND       hwnd;
    HIMAGELIST hImages;

    static const int ICON = 24;
    static const int COUNT = 6;

    void DrawIcons() {
        HDC scr = GetDC(NULL);
        HDC mem = CreateCompatibleDC(scr);
        HBITMAP bmp = CreateCompatibleBitmap(scr, ICON * COUNT, ICON);
        HBITMAP oldBmp = (HBITMAP)SelectObject(mem, bmp);

        RECT all = { 0, 0, ICON * COUNT, ICON };
        HBRUSH bg = CreateSolidBrush(RGB(255, 0, 255));
        FillRect(mem, &all, bg);
        DeleteObject(bg);

        HPEN   pen1 = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
        HPEN   pen2 = CreatePen(PS_SOLID, 2, RGB(0, 0, 0));
        HPEN   penRed = CreatePen(PS_SOLID, 1, RGB(200, 0, 0));
        HPEN   penBlue = CreatePen(PS_SOLID, 1, RGB(0, 0, 200));
        HBRUSH yellow = CreateSolidBrush(RGB(255, 255, 0));
        HBRUSH black = CreateSolidBrush(RGB(0, 0, 0));
        HPEN   oldPen = (HPEN)SelectObject(mem, pen1);
        HBRUSH oldBrush = (HBRUSH)SelectObject(mem, GetStockObject(NULL_BRUSH));

        int ox;

        ox = 0;
        MoveToEx(mem, ox + 12, 4, NULL);  LineTo(mem, ox + 12, 21);
        MoveToEx(mem, ox + 4, 12, NULL);  LineTo(mem, ox + 21, 12);
        SelectObject(mem, black);
        Ellipse(mem, ox + 9, 9, ox + 16, 16);

        ox = ICON;
        SelectObject(mem, pen2);
```

```
        MoveToEx(mem, ox + 4, 20, NULL);  LineTo(mem, ox + 20, 4);

        ox = ICON * 2;
        SelectObject(mem, pen1);
        SelectObject(mem, yellow);
        Rectangle(mem, ox + 3, 6, ox + 21, 19);

        ox = ICON * 3;
        SelectObject(mem, pen2);
        SelectObject(mem, GetStockObject(NULL_BRUSH));
        Ellipse(mem, ox + 3, 5, ox + 21, 19);

        ox = ICON * 4;
        SelectObject(mem, penRed);
        SelectObject(mem, GetStockObject(NULL_BRUSH));
        MoveToEx(mem, ox + 8, 16, NULL);  LineTo(mem, ox + 16, 8);
        Ellipse(mem, ox + 2, 14, ox + 10, 22);
        Ellipse(mem, ox + 14, 2, ox + 22, 10);

        ox = ICON * 5;
        SelectObject(mem, penBlue);
        Rectangle(mem, ox + 9, 3, ox + 21, 15);
        Rectangle(mem, ox + 3, 9, ox + 15, 21);
        MoveToEx(mem, ox + 3, 9, NULL);    LineTo(mem, ox + 9, 3);
        MoveToEx(mem, ox + 15, 9, NULL);   LineTo(mem, ox + 21, 3);
        MoveToEx(mem, ox + 3, 21, NULL);   LineTo(mem, ox + 9, 15);
        MoveToEx(mem, ox + 15, 21, NULL);  LineTo(mem, ox + 21, 15);

        SelectObject(mem, oldPen);
        SelectObject(mem, oldBrush);
        DeleteObject(pen1); DeleteObject(pen2);
        DeleteObject(penRed); DeleteObject(penBlue);
        DeleteObject(yellow); DeleteObject(black);
        SelectObject(mem, oldBmp);
        DeleteDC(mem);
        ReleaseDC(NULL, scr);

        hImages = ImageList_Create(ICON, ICON, ILC_COLOR24 | ILC_MASK, COUNT, 0);
        ImageList_AddMasked(hImages, bmp, RGB(255, 0, 255));
        DeleteObject(bmp);
    }

public:
    Toolbar() : hwnd(NULL), hImages(NULL) {}
    ~Toolbar() { if (hImages) ImageList_Destroy(hImages); }

    bool Create(HWND parent, HINSTANCE hInst) {
        INITCOMMONCONTROLSEX icc = { sizeof(icc), ICC_BAR_CLASSES };
        InitCommonControlsEx(&icc);

        hwnd = CreateWindowExW(0, TOOLBARCLASSNAMEW, NULL,
            WS_CHILD | WS_VISIBLE | TBSTYLE_FLAT | TBSTYLE_TOOLTIPS | CCS_TOP,
            0, 0, 0, 0, parent, (HMENU)(UINT_PTR)ID_TOOLBAR, hInst, NULL);
        if (!hwnd) return false;

        SendMessageW(hwnd, TB_BUTTONSTRUCTSIZE, sizeof(TBBUTTON), 0);

        DrawIcons();
        SendMessageW(hwnd, TB_SETIMAGELIST, 0, (LPARAM)hImages);

        TBBUTTON btn[COUNT] = {};
        const int ids[COUNT] = { IDM_POINT, IDM_LINE, IDM_RECT, IDM_ELLIPSE,
                                 IDM_LINECIRCLE, IDM_CUBE };
        for (int i = 0; i < COUNT; i++) {
            btn[i].iBitmap = i;
            btn[i].idCommand = ids[i];
            btn[i].fsState = TBSTATE_ENABLED;
```

```
            btn[i].fsStyle = BTNS_CHECKGROUP;
        }
        btn[0].fsState |= TBSTATE_CHECKED;
        SendMessageW(hwnd, TB_ADDBUTTONS, COUNT, (LPARAM)btn);
        return true;
    }

    void AutoSize() { SendMessageW(hwnd, TB_AUTOSIZE, 0, 0); }

    void Check(int id) {
        SendMessageW(hwnd, TB_CHECKBUTTON, id, MAKELONG(TRUE, 0));
    }

    static const wchar_t* TipText(int id) {
        switch (id) {
        case IDM_POINT:      return L"Крапка";
        case IDM_LINE:       return L"Лінія";
        case IDM_RECT:       return L"Прямокутник";
        case IDM_ELLIPSE:    return L"Еліпс";
        case IDM_LINECIRCLE: return L"Лінія з кружечками";
        case IDM_CUBE:       return L"Каркас куба";
        }
        return L"";
    }
};

#endif
```

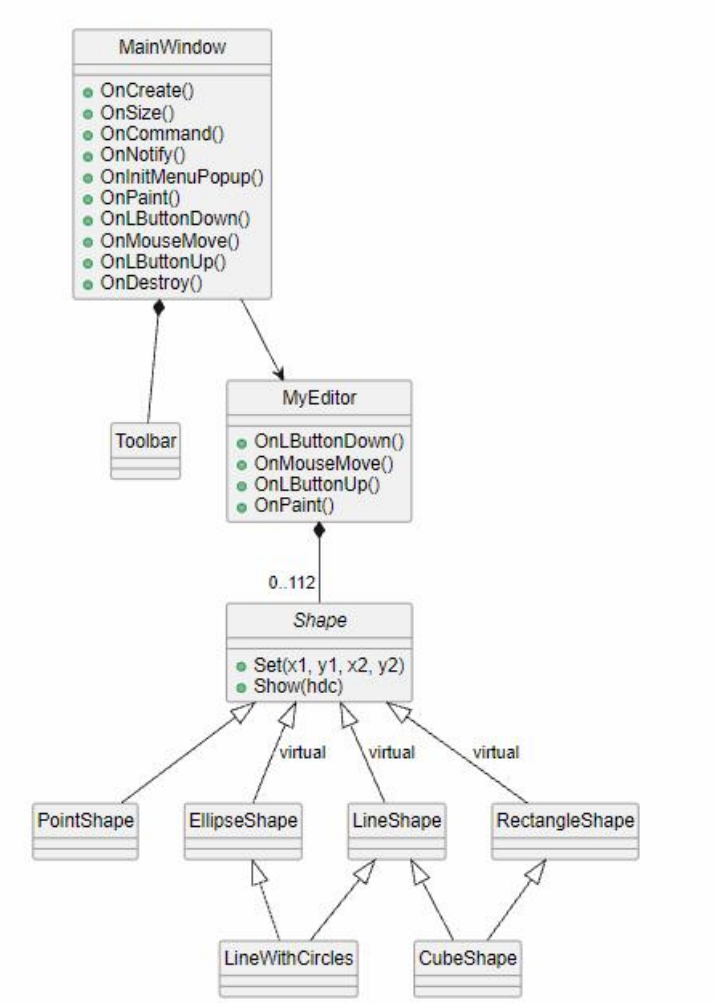
ids.h

```
#ifndef IDS_H
#define IDS_H

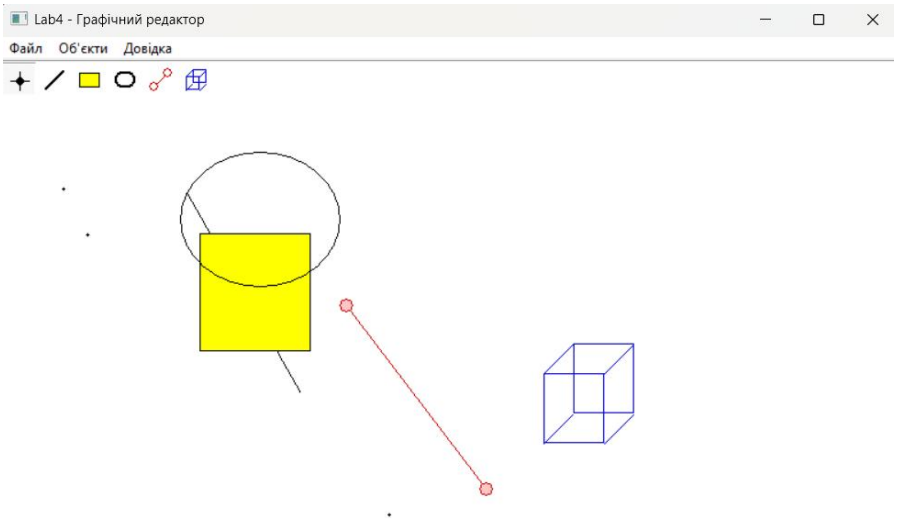
#define IDM_POINT       1001
#define IDM_LINE        1002
#define IDM_RECT        1003
#define IDM_ELLIPSE     1004
#define IDM_LINECIRCLE  1005
#define IDM_CUBE        1006
#define IDM_EXIT        1007
#define IDM_ABOUT       1008
#define ID_TOOLBAR      2001

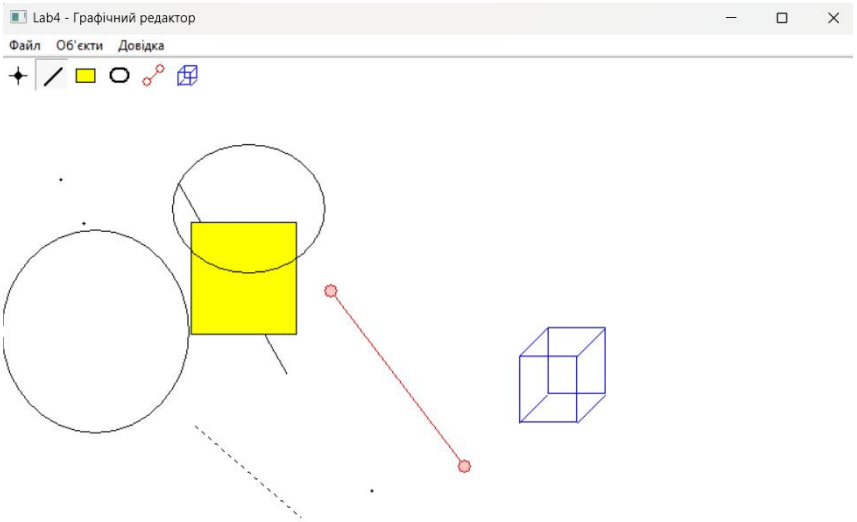
#endif
```

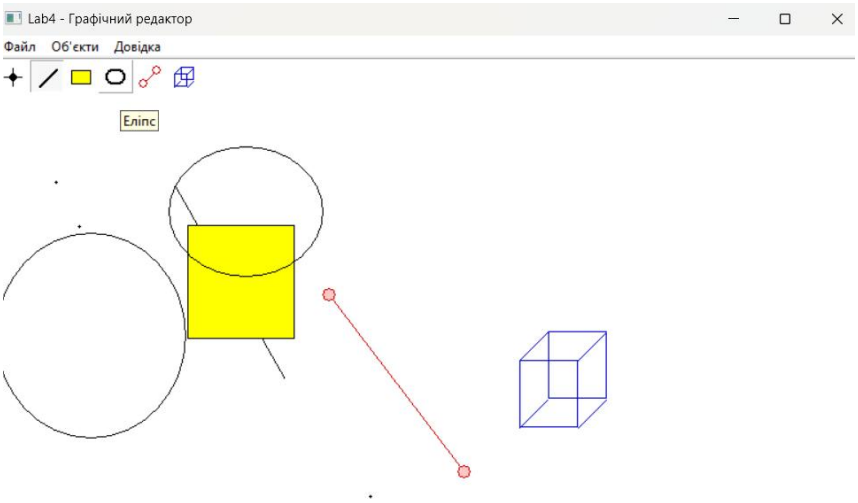
Схеми, діаграми згідно завданню:

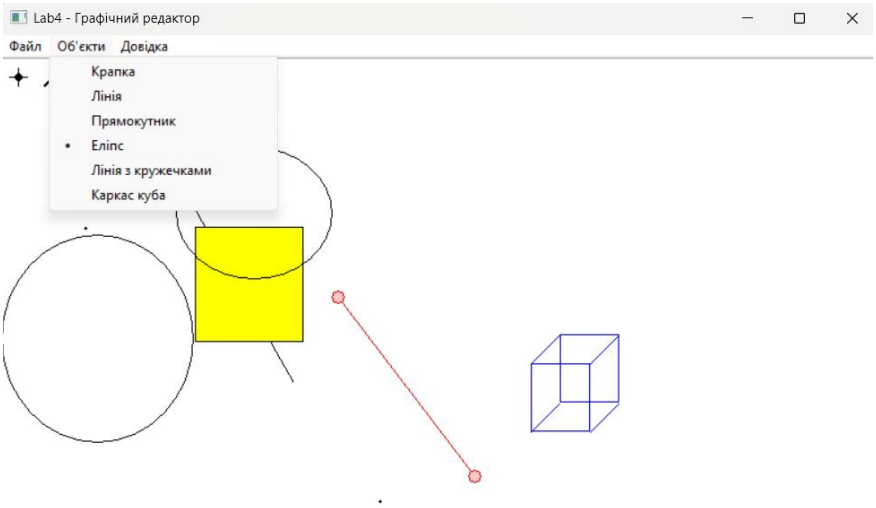


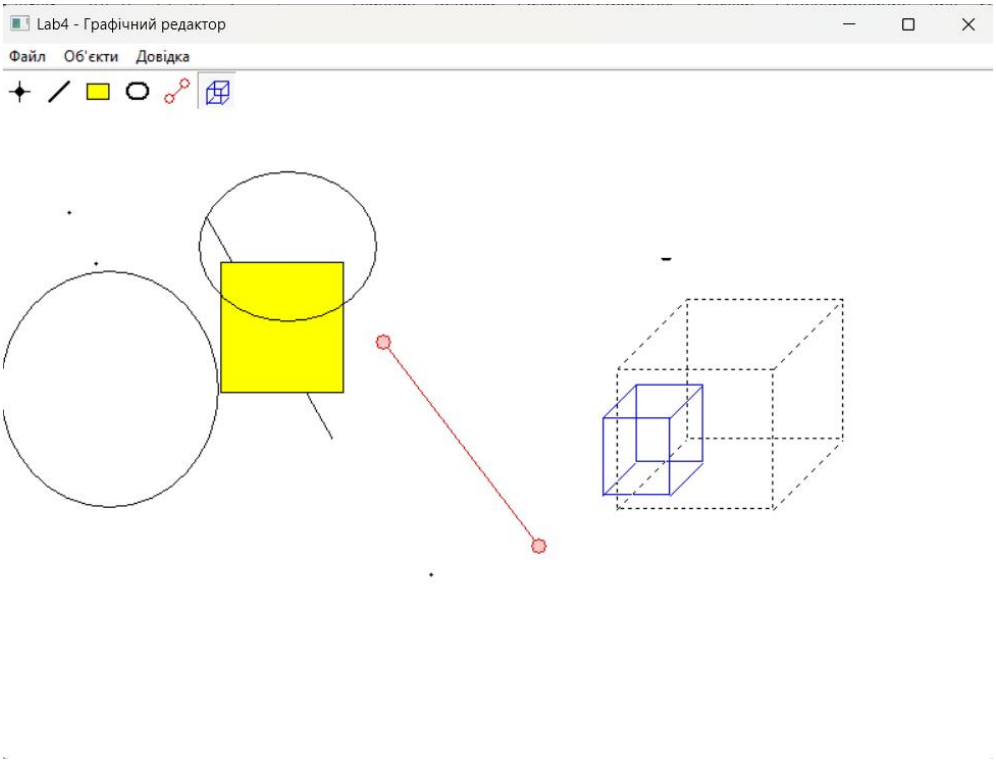
Ілюстрації:




Lab4 - Графічний редактор
Файл
Об'єкти
Довідка


Lab4 - Графічний редактор
Файл
Об'єкти
Довідка
Еліпс





Висновок:

У ході лабораторної роботи №4 я вдосконалила структуру коду графічного редактора на C++ з використанням WinAPI. Логіку зберігання фігур і вводу мишею я винесла з класу вікна в окремий клас MyEditor і, згідно з варіантом №11 (непарний номер), створила його як глобальний статичний об'єкт. Я додала дві нові фігури - лінію з кружечками та каркас куба, і реалізувала їх множинним успадкуванням від LineShape, EllipseShape та RectangleShape, причому методи Show нових класів викликають методи Show базових класів. Щоб уникнути проблем ромбічного успадкування, класи LineShape,

RectangleShape та EllipseShape успадковуються від Shape віртуально, тому в нових фігур є лише одна спільна копія базового класу.

«Гумовий» слід для всіх фігур тепер малюється пунктиром за допомогою тимчасового об'єкта відповідного типу, тому для додавання нової фігури достатньо написати новий клас і додати один рядок у метод CreateShape. Панель Toolbar розширено до шести кнопок з іконками та підказками, які дублюють пункти меню «Об'єкти». У результаті отримано програму, що відповідає вимогам завдання й наочно показує, як поліморфізм і множинне успадкування спрощують розширення коду.